SKALA 4기 · SPRING AI 실습

SpringAI Day1 실습

SKALA

이름 | 김민정
번호 | P282
제출일 | 2026. 08. 18.

목차

1 프로젝트 개요

- 1.1 무엇을 만들었나
- 1.2 코드 구성
- 1.3 아키텍처

2 사전 점검

- 2.1 JDK 확인
- 2.2 실행
- 2.3 키 확인
- 2.4 문서(Swagger UI)

3 Swagger 로 검증

- 3.1 정상
- 3.2 남의 주문
- 3.3 없는 주문
- 3.4 재현성
- 3.5 응답 시간
- 3.6 토큰 사용량

4 실험 — 값을 바꿔보며 감 잡기

- 4.1 온도(temperature)
- 4.2 maxTokens
- 4.3 시스템 프롬프트
- 4.4 정보 누락

5 배운 점

- 5.1 계층 설계
- 5.2 모델 옵션과 재현성
- 5.3 프롬프트 엔지니어링
- 5.4 운영 관점

1. 프로젝트 개요

이 보고서는 Day 1 실습인 "계층 위에 AI 얹기(주문 요약 API)"를 진행한 과정과 결과를 정리했습니다. 1장에서 만든 Controller·Service·Repository 계층 위에, 3장에서 배운 ChatClient를 하나의 엔드포인트(GET /lab1/orders/{id}/summary)로 얹는 것이 이번 실습의 목표였습니다. 새 프로젝트를 만드는 대신, 기존 ch03_chatclient 프로젝트에 day1(lab1) 패키지를 추가하는 방식으로 진행했습니다.

1.2 코드 구성

패키지 / 파일	역할
domain/Order, OrderStatus, OrderNotFoundException	1장에서 쓰던 계층 그대로입니다. cost처럼 내부에서만 쓰는 필드는 응답에 나가지 않습니다.
repository/OrderRepository	findByIdAndOwnerId로 권한까지 쿼리 안에서 함께 검사합니다. 조회 후 자바 코드에서 소유자를 비교하지 않습니다.
config/Lab1AiConfig	요약 전용 ChatClient 빈입니다. temperature 0, maxTokens 120을 빈에 고정해서, 호출부마다 옵션이 달라지는 일을 막았습니다.
service/OrderSummaryService	주문 조회, AI 호출, 실패 시 폴백을 전부 이 클래스 하나에서 처리합니다. 이번 실습에서 가장 공들인 부분입니다.
web/OrderSummaryController	ChatClient를 전혀 모릅니다. 서비스만 바라봅니다.
web/Lab1ExceptionHandler	예외를 응답으로 바꾸는 일을 한 곳에 모아뒀습니다 (404 / 503).

1.3 아키텍처

요청 한 번이 어떤 순서로 계층을 지나는지 정리하면 아래와 같습니다.

- 1. web/OrderSummaryController가 HTTP 요청을 받습니다. 이 계층은 ChatClient를 모릅니다.
- 2. service/OrderSummaryService가 orders.findByIdAndOwnerId로 권한과 존재 여부를 한 번에 확인합니다.
- 3-a. repository/OrderRepository → H2(Order)로 주문 데이터를 가져옵니다.
- 3-b. config/Lab1AiConfig가 구성한 ChatClient(summaryChatClient) → OpenAI(gpt-4o-mini)로 한 문장 요약을 받아옵니다.

- 4. 정상이면 SummaryResponse(200)를 돌려주고, AI 호출만 실패하면 상품·상태로 만든 문구로 대신하며, 주문을 못 찾으면 Lab1ExceptionHandler가 404 또는 503으로 응답합니다.

2. 사전 점검

실습을 시작하기 전에, 환경이 준비됐는지 여섯 가지를 먼저 확인했습니다. 여기서 막히면 나중에 환경 문제인지 코드 문제인지 헷갈리기 쉬워서, 코드를 건드리기 전에 미리 통과시켜두는 순서입니다.

확인할 것	명령 · 화면	통과 기준
JDK	java -version	21이 나온다
프로젝트	VS Code로 **실습 폴더** 폴더 열기	권장 확장 설치 알림 → 설치
실행	F5 또는 ./gradlew bootRun	기동 로그에 포트 8080
키	echo $OPENAI_API_KEY	**sk-** 로 시작 — 없으면 실습을 시작할 수 없다
계층 예제	/ch02/orders/12345?userId=user1	200 + JSON (키 없이도 동작)
문서	/swagger-ui.html	그룹 세 개가 보인다

그림 1. 사전 점검 체크리스트 — JDK · 프로젝트 열기 · 실행 · 키 · 계층 예제 · 문서

2.1 JDK 확인

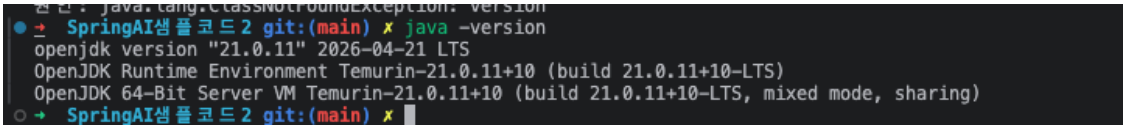


그림 2. java -version 실행 결과

openjdk version "21.0.11" (2026-04-21 LTS)로 JDK 21 버전임을 확인했습니다.

2.2 실행

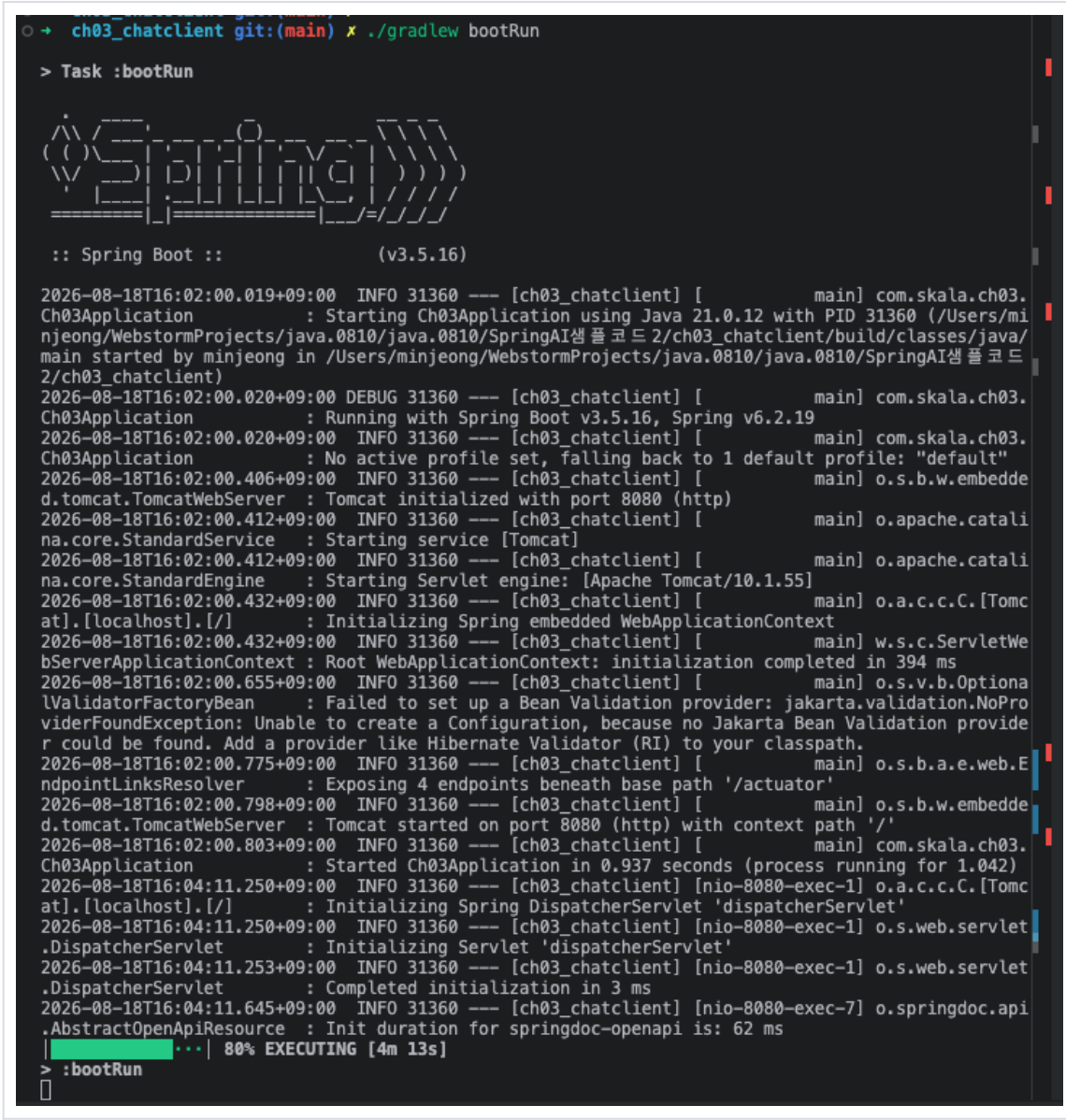


그림 3. ./gradlew bootRun 실행 로그

bootRun 로그를 보면 Spring Boot 3.5.16 배너와 함께 ch03_chatclient 앱이 8080 포트에 뜬 것을 확인했습니다.

2.3 키 확인

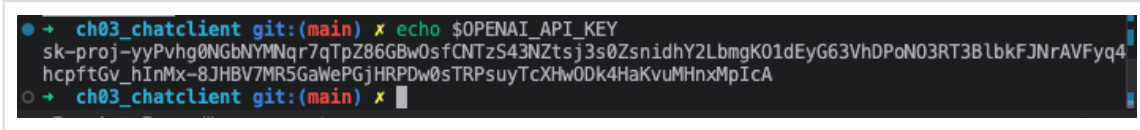


그림 4. echo $OPENAI_API_KEY 실행 결과

출력이 sk-proj-로 시작하는 걸 보고 키가 잘 로드된 걸 확인했습니다.

2.4 문서 (Swagger UI)

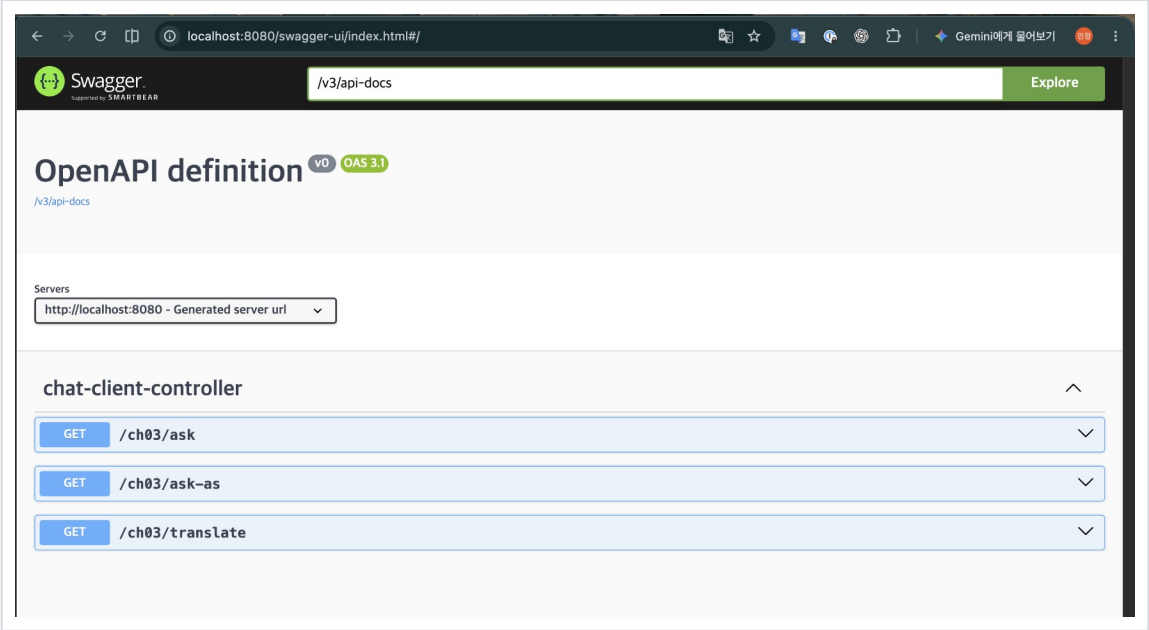


그림 5. Swagger UI 초기 화면 — chat-client-controller 그룹

캡처는 chat-client-controller 그룹(ch03의 /ask · /ask-as · /translate)을 보여줍니다. lab1/orders 그룹은 이 화면보다 아래쪽, day1 패키지가 스캔된 뒤에 별도로 잡힙니다. 뒤에서(3장) 실제로 호출되는 걸 보면 정상 등록된 걸 알 수 있습니다.

3. Swagger 로 검증

완성한 GET /lab1/orders/{orderId}/summary 엔드포인트를 Swagger UI의 Try it out으로 직접 호출하면서, 정상·권한·부재·재현성·응답시간·토큰까지 여섯 가지를 확인했습니다.

눌러 볼 것	입력	결과
정상	12345 / user1	200 · 한 문장 요약 ("무선 이어폰이 배송 중이며 8월 20일 도착 예정"이라는 문장 반환)
남의 주문	99999 / user1	404 · 99999는 user2 소유라 조회되지 않음
없는 주문	00000 / user1	404 · 남의 주문과 완전히 같은 메시지 (존재 여부를 알리지 않음)
재현성	같은 입력으로 3회	문장이 토씨 하나 없이 동일 (temperature 0)
응답 시간	Request duration 확인	1000~1900ms대 — 대부분 모델 호출 구간
토큰	/actuator/metrics/ gen_ai.client.token.usage	호출할 때마다 값이 증가 (1620 → 1890)

3.1 정상 (12345 / user1)

```
curl -X 'GET' 'http://localhost:8080/lab1/orders/12345/summary?userId=user1'


// 200 OK
{"orderId":"12345","summary":"주문번호 12345의 무선 이어폰은 현재 배송 중이며, 도착 예정일은 2026년 8월 20일입니다."}
```

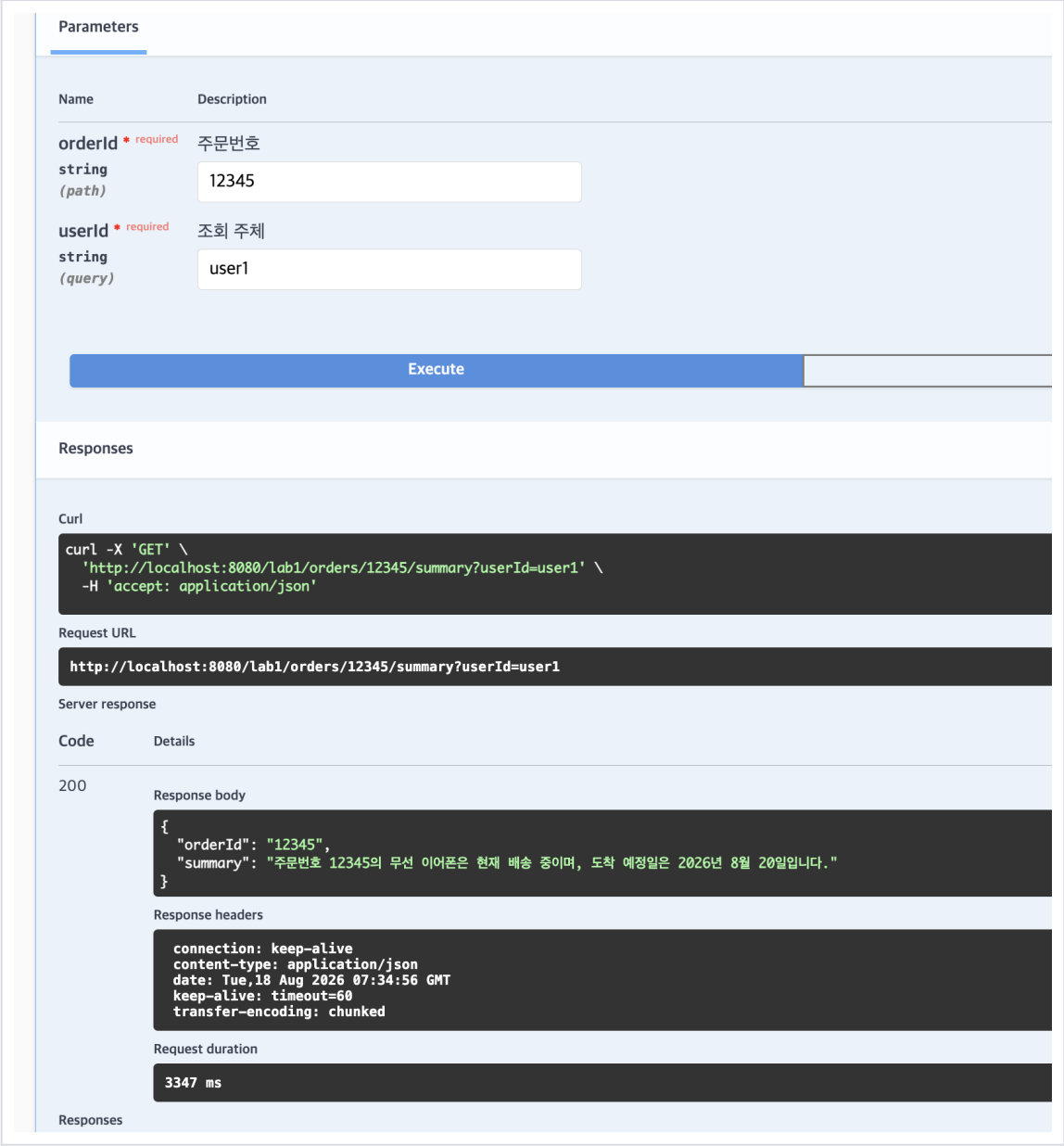


그림 6. 정상 응답 — Swagger UI 실행 화면 (Request duration 3347ms)

첫 호출이다 보니 Request duration이 3347ms로, 뒤에서(3.4, 3.5) 잰 1000~1900ms대보다 눈에 띄게 느렸습니다. 같은 프롬프트라도 콜드 스타트 직후 첫 호출은 원래 느린 편이라, 응답 시간을 비교할 때는 워밍업 이후 값을 기준으로 삼는 게 좋습니다.

3.2 남의 주문 (99999 / user1)

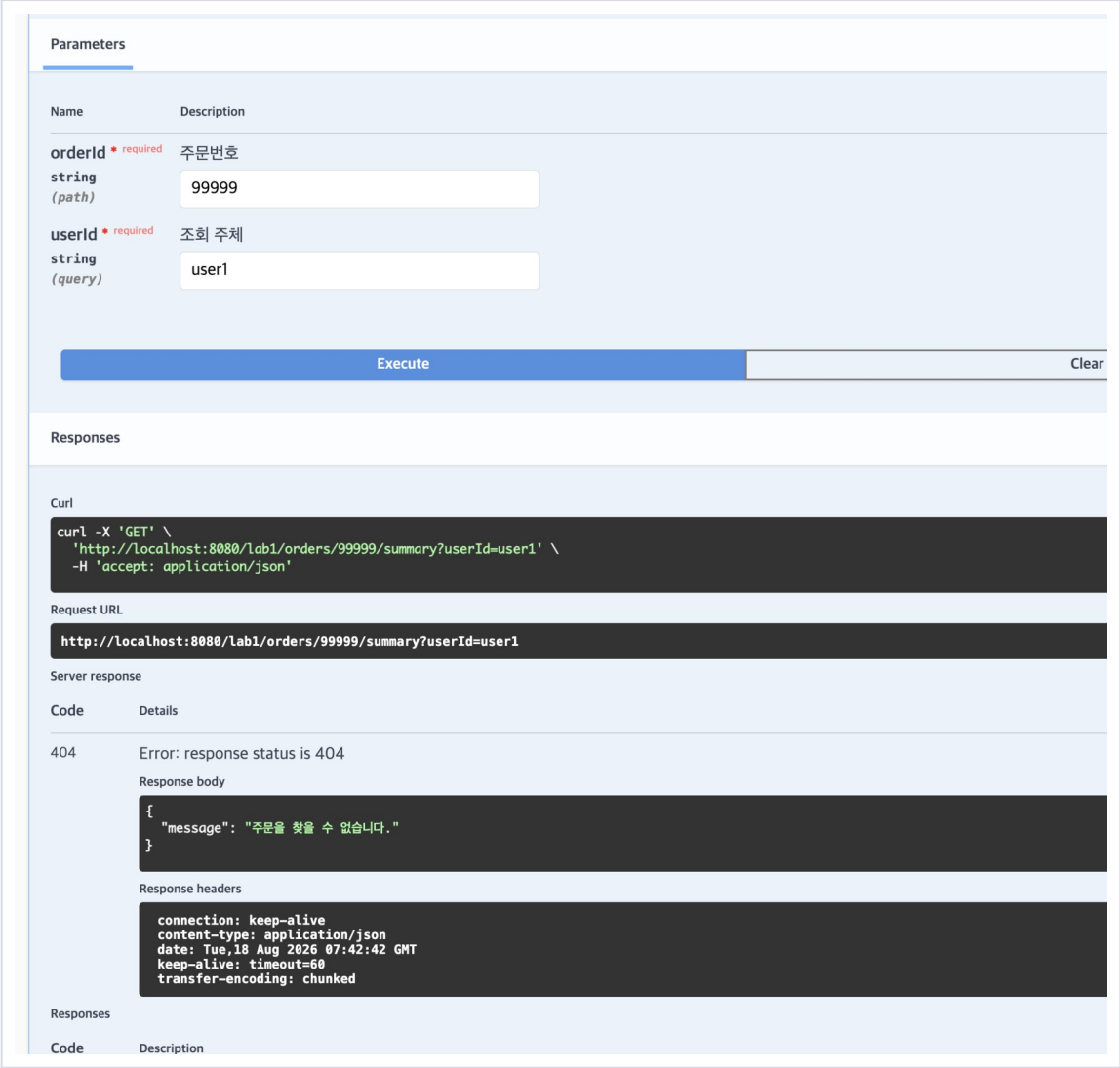


그림 7. 남의 주문 조회 — 404

99999는 user2 소유라 404 {"message":"주문을 찾을 수 없습니다."}를 받았습니다. "주문이 아예 없는 경우"(3.3)와 메시지가 토씨 하나 다르지 않은데, 이유를 굳이 구분해서 알려주지 않는 게 의도된 설계입니다. "남의 주문이라 못 봄"과 "애초에 없음"을 구분해서 응답하면, 존재하는 주문번호를 무차별로 훑는 공격에 힌트를 주게 됩니다.

3.3 없는 주문 (00000 / user1)

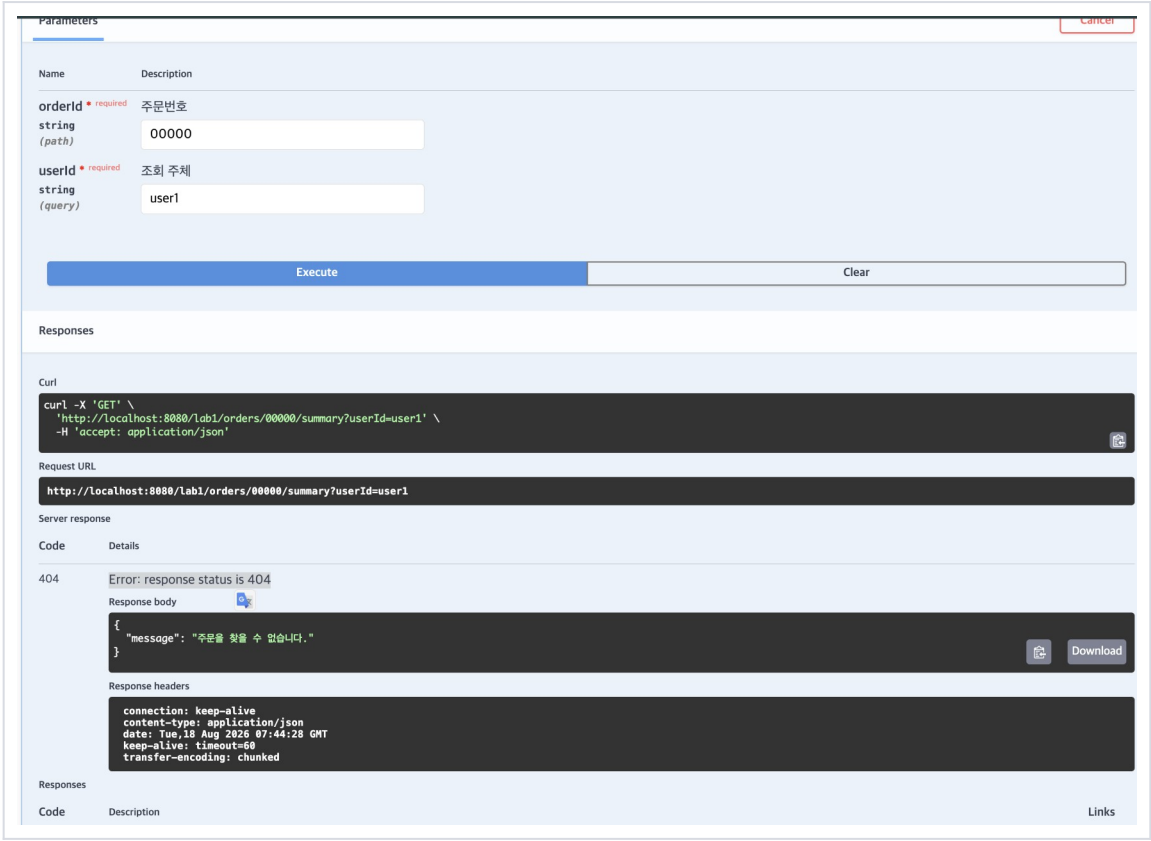


그림 8. 없는 주문 조회 — 404

3.2(남의 주문)와 코드·메시지가 완전히 같습니다. Lab1ExceptionHandler가 OrderNotFoundException 하나로 두 상황을 묶어서 응답하기 때문입니다. 두 캡처를 나란히 보면 그 설계가 실제로 지켜지고 있다는 걸 알 수 있습니다.

3.4 재현성 (같은 요청 3회)

orderId * required 주문번호
string (path) 12345

userId * required 조회 주체
string (query) user1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/lab1/orders/12345/summary?userId=user1' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/lab1/orders/12345/summary?userId=user1
```

Server response

Code Details

200

Response body

```
{
  "orderId": "12345",
  "summary": "주문번호 12345의 무선 이어폰은 현재 배송 중이며, 도착 예정일은 2026년 8월 20일입니다."
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue,18 Aug 2026 08:00:57 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Request duration

```
1284 ms
```

Responses

그림 9. 재현성 확인 1회차 (1284ms)

orderId * required 주문번호
string (path) 12345

userId * required 조회 주체
string (query) user1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/lab1/orders/12345/summary?userId=user1' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/lab1/orders/12345/summary?userId=user1
```

Server response

Code Details

200

Response body

```
{
  "orderId": "12345",
  "summary": "주문번호 12345의 무선 이어폰은 현재 배송 중이며, 도착 예정일은 2026년 8월 20일입니다."
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue,18 Aug 2026 08:01:09 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Request duration

```
1205 ms
```

Responses

Code Description Links

그림 10. 재현성 확인 2회차 (1205ms)

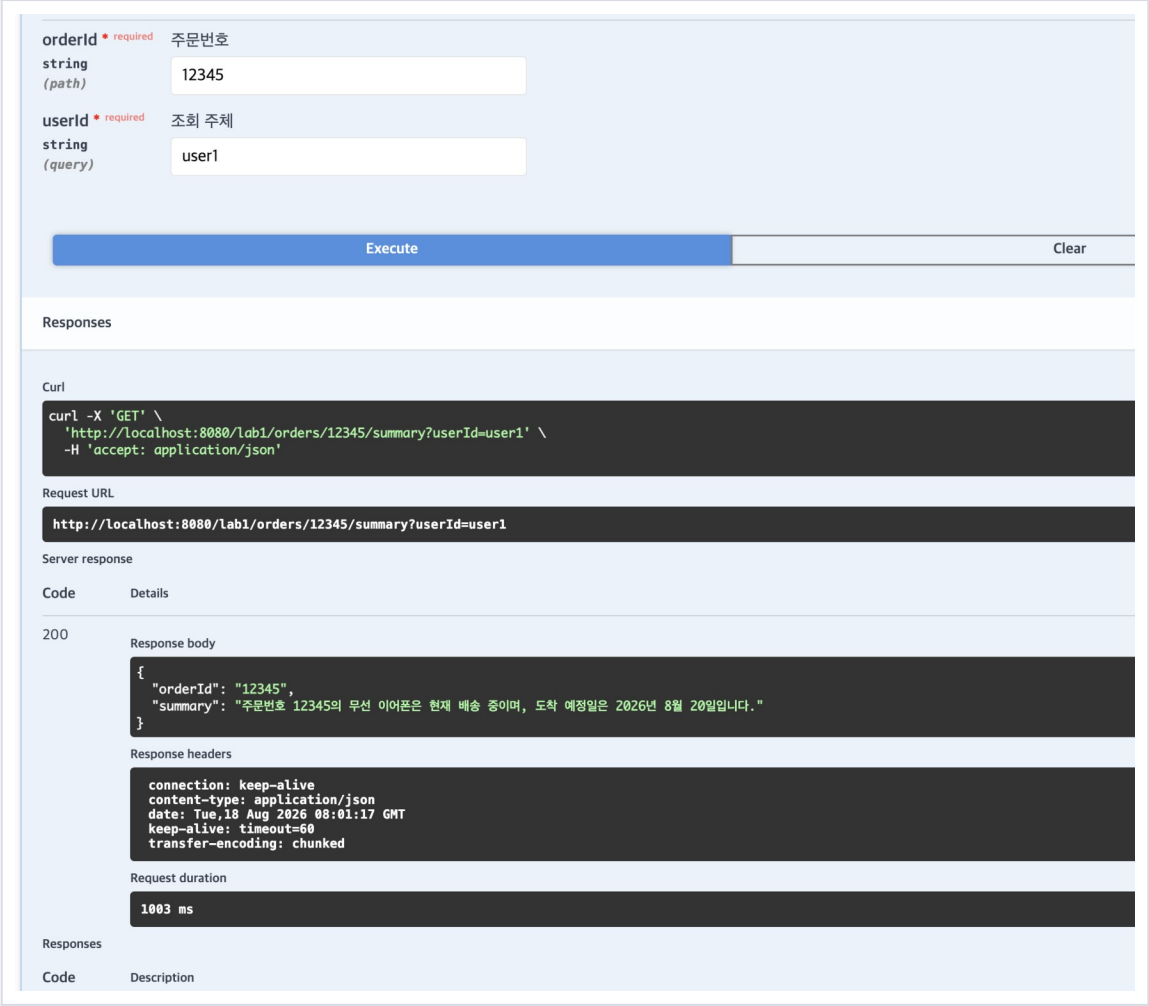


그림 11. 재현성 확인 3회차 (1003ms)

같은 요청을 3번 보냈는데 summary 문장이 매번 동일했습니다. temperature를 0으로 해두었기 때문에 같은 요청에도 모두 같은 응답이 나온다는 것을 확인할 수 있었습니다. Lab1AiConfig에서 요약 전용 클라이언트의 온도를 0으로 고정한 것도 이런 이유 때문입니다. 호출부마다 온도를 다르게 두면, 같은 주문을 요약해도 매번 다른 문장이 나옵니다. 세 번의 request duration은 각각 1284ms · 1205ms · 1003ms로, 3.1의 첫 호출(3347ms)보다 뚜렷하게 짧았습니다. 텍스트뿐 아니라 속도까지 안정되는 게 반복 호출의 특징입니다.

3.5 응답 시간



그림 12. 응답 시간 확인 (1837ms)

Request duration은 1837ms — 3.4에서 반복 호출했을 때의 1003~1284ms 구간보다 조금 깁니다. 모델 호출 지연은 네트워크 상태에 따라 요청마다 수백 ms씩 흔들린다는 걸 보여주는 수치입니다.

3.6 토큰 사용량

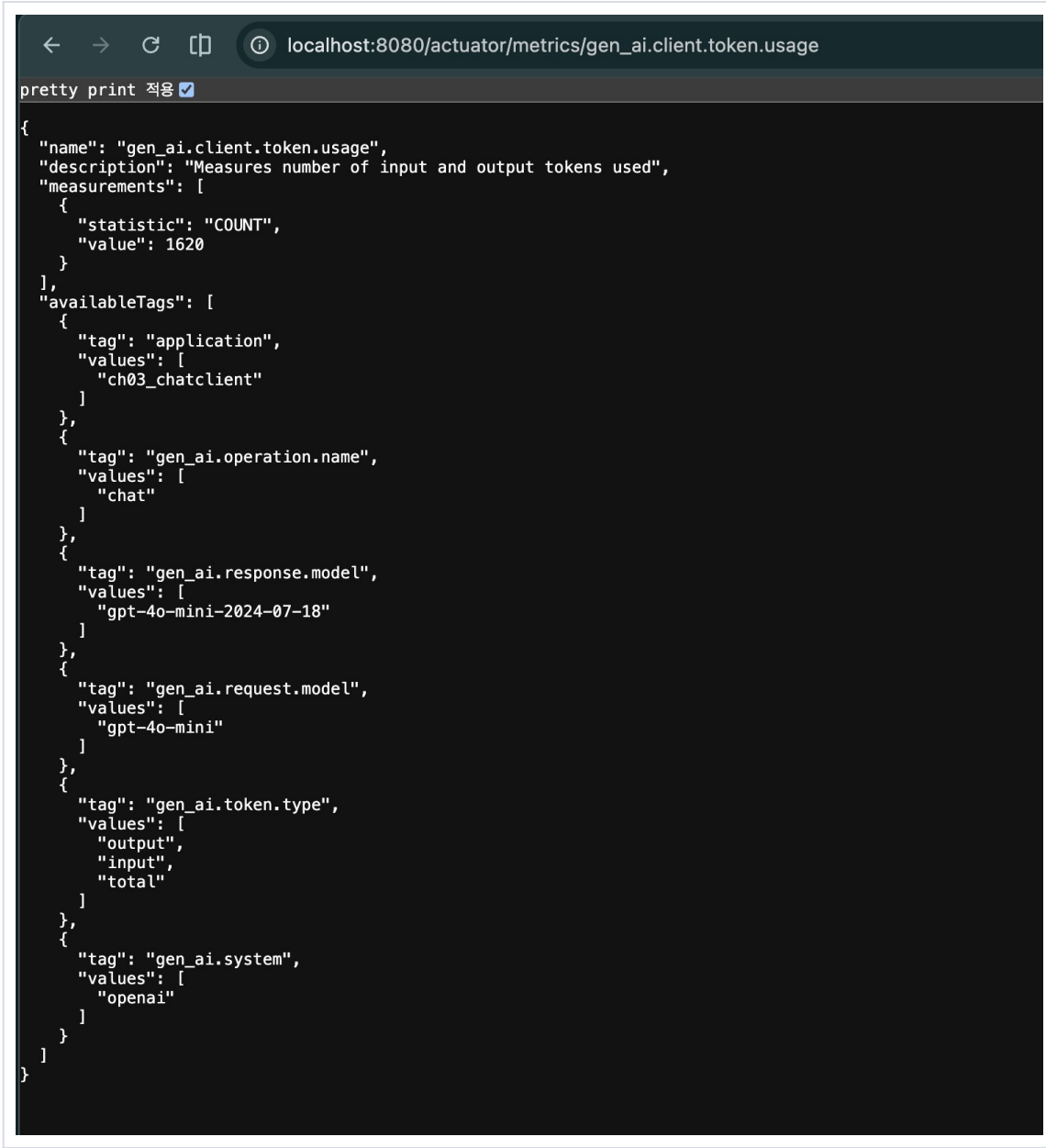


그림 13. /actuator/metrics/gen_ai.client.token.usage — 1620

pretty print 적용 ☑


```
{
  "name": "gen_ai.client.token.usage",
  "description": "Measures number of input and output tokens used",
  "measurements": [
    {
      "statistic": "COUNT",
      "value": 1890
    }
  ],
  "availableTags": [
    {
      "tag": "application",
      "values": [
        "ch03_chatclient"
      ]
    },
    {
      "tag": "gen_ai.operation.name",
      "values": [
        "chat"
      ]
    },
    {
      "tag": "gen_ai.response.model",
      "values": [
        "gpt-4o-mini-2024-07-18"
      ]
    },
    {
      "tag": "gen_ai.request.model",
      "values": [
        "gpt-4o-mini"
      ]
    },
    {
      "tag": "gen_ai.token.type",
      "values": [
        "output",
        "input",
        "total"
      ]
    },
    {
      "tag": "gen_ai.system",
      "values": [
        "openai"
      ]
    }
  ]
}
```


그림 14. 같은 엔드포인트 — 1890 (호출 후)

1620 → 1890으로 토큰 사용량이 늘어난 걸 확인했습니다. availableTags의 application 값이 ch03_chatclient인 것도 함께 보입니다. 이 지표는 요청 한 번의 토큰이 아니라 앱이 뜬 이후 누적된 COUNT 값이라서, 호출할수록 계속 늘어납니다.

4. 실험 — 값을 바꿔보며 감 잡기

Lab1AiConfig와 프롬프트를 하나씩 바꿔가며, 옵션 하나가 결과에 어떤 영향을 주는지 직접 확인했습니다.

바꿔 볼 것	어떻게	결과
온도	0 → 0.9로 바꿔 호출	문장이 매번 다르게 나옴 (순서·날짜 표기 방식까지 달라짐)
maxTokens	120 → 20	문장이 중간에서 끊김 (닫는 따옴표도 없이 잘림)
시스템 프롬프트	한 문장 → 세 문장으로	지시한 그대로 정확히 세 문장으로 나뉘어 응답
정보 누락	프롬프트에서 eta를 빼 봄	거절 문구 없이 조용히 생략 → "없음" 명시 + 안내 지시 추가로 개선

4.1 온도 (temperature 0 → 0.9)

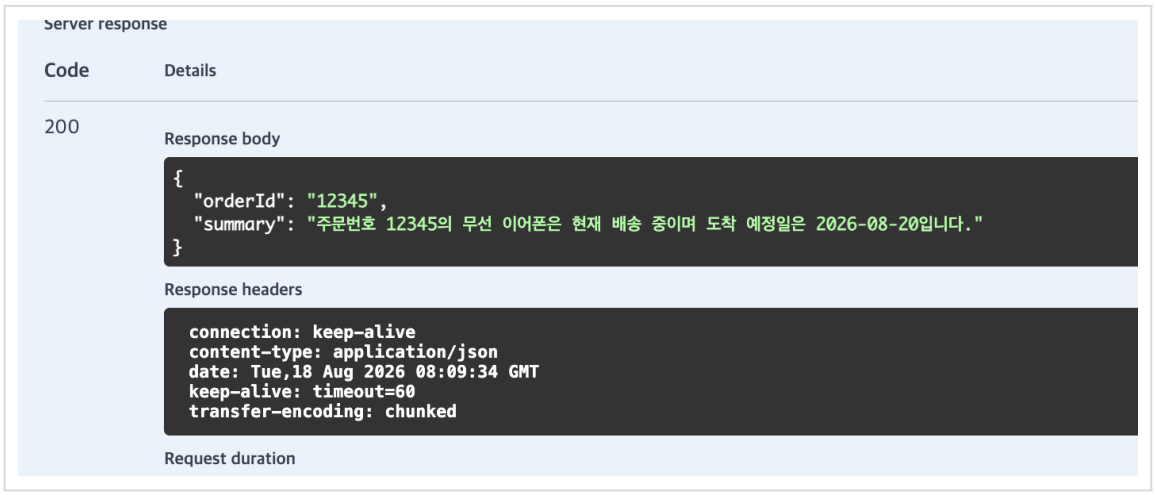


그림 15. temperature 0.9 — 1회차

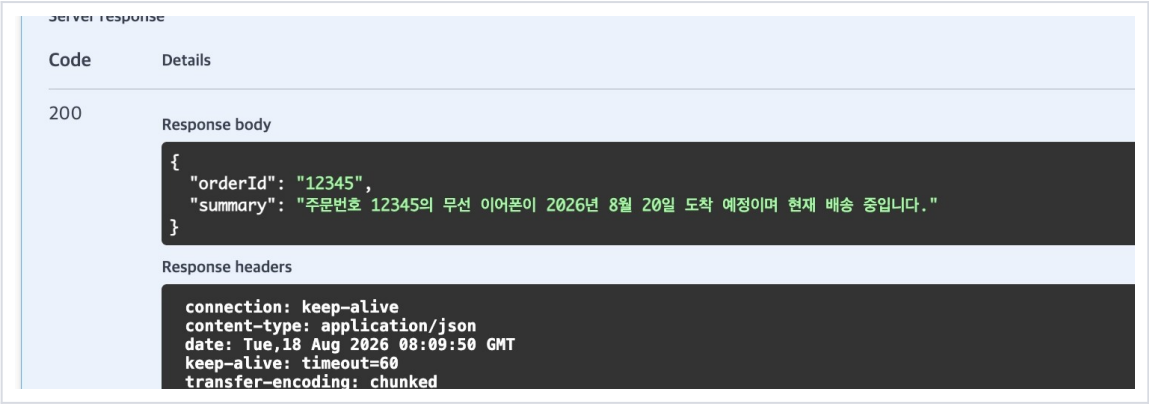


그림 16. temperature 0.9 — 2회차

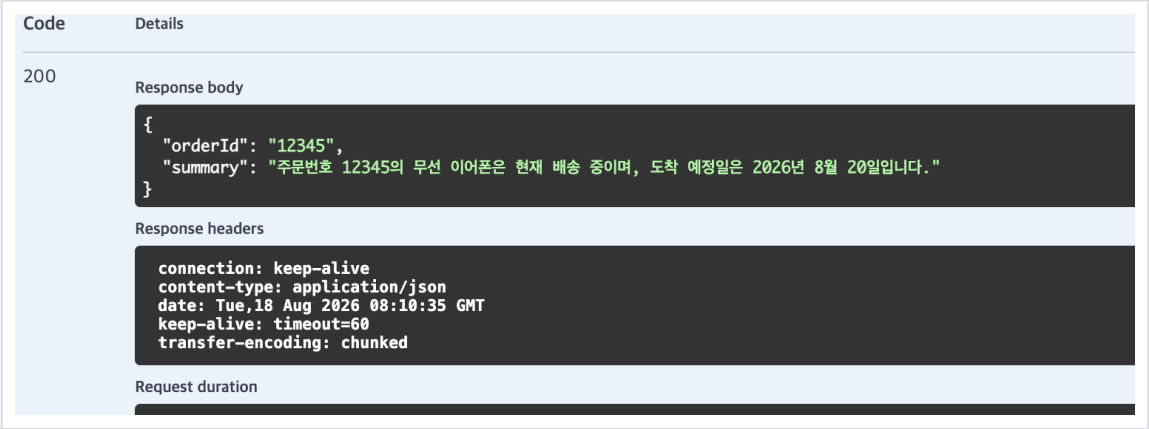


그림 17. temperature 0.9 — 3회차

응답이 조금씩 다르게 나오는 걸 확인했습니다. 캡처 세 개를 비교해보면 "배송 중이며 도착 예정일은 2026-08-20" / "이어폰이 8월 20일 도착 예정이며 현재 배송 중"처럼 문장 순서와 날짜 표기 방식(2026-08-20 vs 2026년 8월 20일)까지 매번 달라집니다. temperature 0일 때는 토씨 하나 안 바뀌던 것과 뚜렷하게 대비됩니다.

4.2 maxTokens (120 → 20)

```
@Configuration
class Lab1AiConfig {

    /** 요약 전용 클라이언트 — 매번 같은 결과가 나와야 하므로 temperature 0, 길게 쓸 이유가 없으므로 토
    @Bean
    ChatClient summaryChatClient(ChatClient.Builder builder) {
        return builder
                .defaultSystem("""
                        너는 이커머스 주문 상담 도우미다.
                        주어진 주문 정보만 사용해 한국어 한 문장으로 요약한다.
                        추측하지 않는다. 정보가 부족하면 "정보가 부족합니다"라고 답한다.
                        """)
                .defaultOptions(ChatOptions.builder()
                        .temperature(0.0)
-                       .maxTokens(120)
+                       .maxTokens(20)
                        .build())
                .build();
    }
}
```

그림 18. Lab1AiConfig 코드 변경 — maxTokens 120 → 20

Code

Details

200

Response body

```
{
  "orderId": "12345",
  "summary": "주문번호 12345의 무선 이어폰은 현재 배송 중이며, 도착 예정"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue,18 Aug 2026 08:13:33 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

그림 19. maxTokens 20 적용 결과 — 문장이 잘림

문장이 중간에서 끊기는 걸 확인했습니다. 실제 응답이 "...도착 예정"에서 닫는 따옴표도 없이 그대로 잘려 있었습니다. maxTokens 20은 모델이 문장을 마무리하기도 전에 강제로 끊어버리는 값이라는 걸 캡처로 확인할 수 있습니다.

4.3 시스템 프롬프트 (한 문장 → 세 문장)

```
/** 요약 전용 클라이언트 — 매번 같은 결과가 나와야 하므로 temperature 0, 길게 쓸 이유가 없
@Bean
ChatClient summaryChatClient(ChatClient.Builder builder) {
    return builder
            .defaultSystem("""
                    너는 이커머스 주문 상담 도우미다.
                    주어진 주문 정보만 사용해 한국어 한 문장으로 요약한다.
                    주어진 주문 정보만 사용해 한국어 세 문장으로 요약한다.
                    추측하지 않는다. 정보가 부족하면 "정보가 부족합니다"라고 답한다.
                    """)
            .defaultOptions(ChatOptions.builder()
                    .temperature(0.0)
                    .maxTokens(120)
                    .build())
            .build();
}
```

그림 20. 시스템 프롬프트 코드 변경 — "한 문장" → "세 문장으로"

Curl

```
curl -X 'GET' \
  'http://localhost:8080/lab1/orders/12345/summary?userId=user1' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/lab1/orders/12345/summary?userId=user1
```

Server response

Code	Details
200	Response body

```
{
  "orderId": "12345",
  "summary": "주문번호 12345의 상품은 무선 이어폰입니다. 현재 상태는 배송 중입니다. 도착 예정일은 2026년 8월 20일입니다."
}
```

Response headers

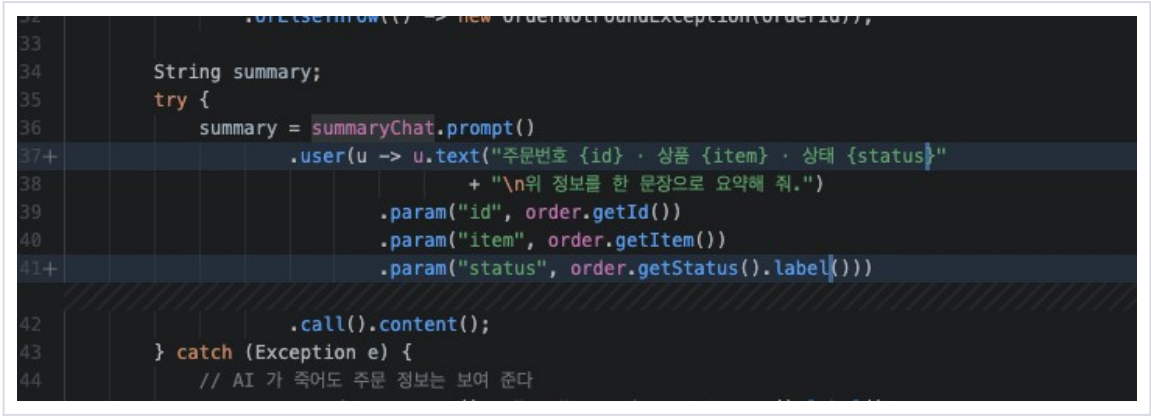
```
connection: keep-alive
content-type: application/json
date: Tue,18 Aug 2026 08:27:23 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Request duration

그림 21. 세 문장으로 나뉜 응답

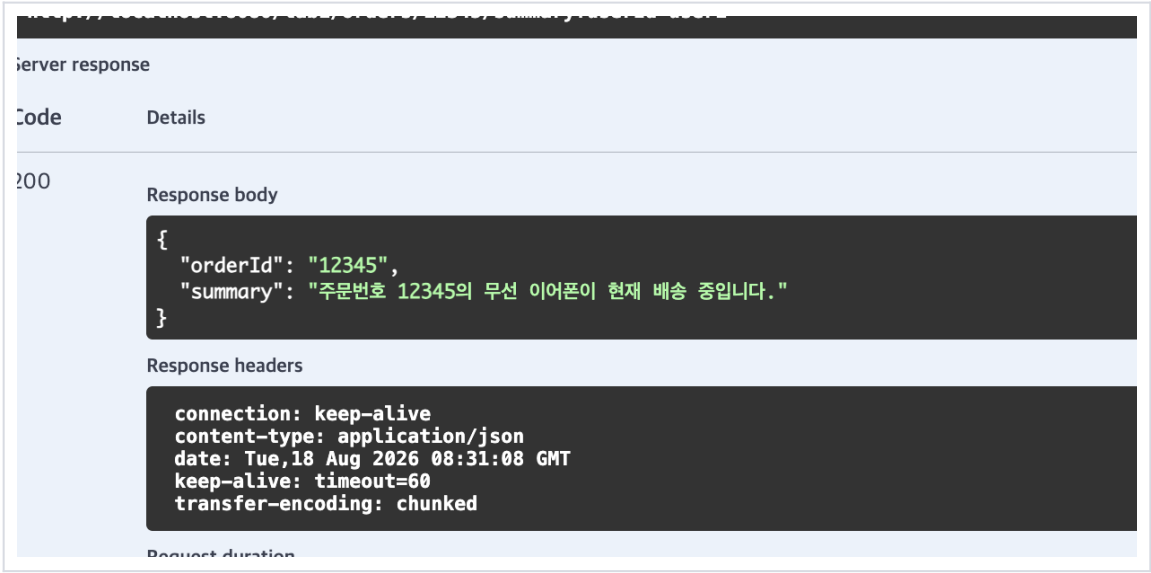
"주문번호 12345의 상품은 무선 이어폰입니다. 현재 상태는 배송 중입니다. 도착 예정일은 2026년 8월 20일입니다."처럼, 정보 하나당 문장 하나씩 정확히 세 문장으로 나뉘었습니다. 지시를 숫자로 명확하게 줄수록 모델이 형식을 더 잘 지킨다는 걸 보여주는 결과입니다.

4.4 정보 누락 (eta 파라미터 제거)



```
String summary;
try {
    summary = summaryChat.prompt()
            .user(u -> u.text("주문번호 {id} · 상품 {item} · 상태 {status}"
                    + "\n위 정보를 한 문장으로 요약해 줘.")
                    .param("id", order.getId())
                    .param("item", order.getItem())
                    .param("status", order.getStatus().label()))
            .call().content();
} catch (Exception e) {
    // AI 가 죽어도 주문 정보는 보여 준다
```

그림 22. 프롬프트에서 eta 파라미터 제거



Server response

Code Details

200

Response body

```
{
  "orderId": "12345",
  "summary": "주문번호 12345의 무선 이어폰이 현재 배송 중입니다."
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue,18 Aug 2026 08:31:08 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

그림 23. eta 없이 호출한 결과 — 거절 문구 없이 조용히 생략

먼저 eta 파라미터를 프롬프트에서 빼는 시도를 해봤습니다. 빼보니 도착 예정 정보만 조용히 안 나올 뿐, 이에 대한 별도 처리는 없었습니다. 시스템 프롬프트에 "정보가 부족하면 정보가 부족합니다라고 답한다"는 거절 규칙을 넣어뒀는데도, 모델은 거절 문구 없이 그냥 생략해버린 것입니다.



```
        String eta = order.getEta() != null ? order.getEta().toString() : "없음";

        String summary;
        try {
            summary = summaryChat.prompt()
                    .user(u -> u.text("주문번호 {id} · 상품 {item} · 상태 {status} · 도착예정 {eta}"
                                    + "\n위 정보를 한 문장으로 요약해 줘. 도착예정이 \"없음\"이면 도착 예정 정보가 없다고 알려줘.")
                            .param("id", order.getId())
                            .param("item", order.getItem())
                            .param("status", order.getStatus().label())
                            .param("eta", eta))
                    .call().content();
        } catch (Exception e) {
            // AI 가 죽어도 주문 정보는 보여 준다
            summary = order.getItem() + " · " + order.getStatus().label();
        }

        return new SummaryResponse(order.getId(), summary);
    }
```

그림 24. eta 를 "없음" 문자열로 명시하고, 안내 지시를 추가한 개선 버전

이 상태로는 부족하다고 판단해서, eta 정보가 없을 때는 도착 예정 정보가 없다는 문구가 뜨도록 직접 추가로 구현했습니다. eta를 프롬프트에서 아예 빼는 대신 값을 "없음"으로 명시하고, "도착예정이 없음이면 도착 예정 정보가 없다고 알려줘"라는 지시를 시스템 프롬프트에 추가했습니다.

5. 배운 점

5.1 계층 설계

- AI는 새로운 계층이 아니라, 기존 계층(Controller-Service-Repository) 위에 얹히는 관심사 하나라는 것을 배웠습니다. 계층을 먼저 세우고 AI를 그 위에 얹는 순서를 지키면, 나중에 모델이나 프롬프트를 바꿔도 웹 계층은 그대로 남는다는 것도 알게 되었습니다.
- 강의에서 설명해주신 "컨트롤러는 ChatClient를 몰라야 한다"는 원칙이 실제 코드에서 그대로 활용되는 것을 확인할 수 있었습니다. AiConfig가 빈을 구성하고, Service가 업무 흐름과 AI 호출·폴백을 모두 쥐는 구조였습니다.
- 권한 조건(findByIdAndOwnerId)은 쿼리 안에 넣고, "남의 주문"과 "없는 주문"의 응답을 일부러 똑같이 만들어서 정보가 새어나가지 않게 설계하는 습관을 이번 실습을 통해 익혔습니다.

5.2 모델 옵션과 재현성

- temperature 0은 "매번 같은 답"을, 0.9는 "매번 다른 표현"을 만든다는 것을 3회 반복 호출과 실험으로 직접 확인할 수 있었습니다. 요약처럼 일관성이 중요한 기능은 temperature를 0으로 빈에 고정해야 한다는 것을 배웠습니다.
- maxTokens는 비용 상한이면서 동시에 품질 리스크가 될 수 있다는 것을 알게 되었습니다. 상한을 너무 낮게 잡으면 문장이 채 끝나기도 전에 끊겨버리기 때문에, 상한을 정할 때는 "가장 긴 정상 응답이 몇 토큰인지"부터 가늠해봐야 한다는 것을 배웠습니다.
- 첫 호출(3347ms)이 이후 반복 호출(1000~1900ms대)보다 눈에 띄게 느리다는 것을 수치로 확인했습니다. 응답 시간을 재거나 비교할 때는 콜드 스타트 영향을 뺀 워밍업 이후 값을 써야 한다는 것을 알게 되었습니다.

5.3 프롬프트 엔지니어링

- 시스템 프롬프트의 지시를 "한 문장" → "세 문장으로"처럼 숫자로 구체화하면 모델이 형식을 훨씬 더 정확하게 지킨다는 것을 배웠습니다.
- 가장 크게 배운 것은 정보 누락 처리였습니다. 프롬프트에서 파라미터 자체를 빼버리면, 시스템 프롬프트에 "정보 부족 시 거절" 규칙을 넣어뒀더라도 모델이 그 사실을 알아채지 못하고 그냥 생략해버린다는 것을 알게 되었습니다. "정보가 없다"는 건 모델이 알아서 추론할 대상이 아니라, 프롬프트가 명시적으로 알려줘야 하는 사실이라는 것을 실험을 통해 배웠습니다.
- 강의에서 강조해주신 "AI가 실패해도 화면은 살아야 한다"는 원칙이 실제로 어떻게 구현되는지 확인할 수 있었습니다. OrderSummaryService는 모델 호출을 try/catch로 감싸서, 실패하면 상품·상태만으로 만든 폴백 문구를 돌려줍니다. 요약은 부가 정보고, 주문 정보 자체는 AI 성패와 상관없이 나가야 한다는 1장의 원칙이 코드로 활용되는 것을 확인할 수 있었습니다.

5.4 운영 관점

- gen_ai.client.token.usage 같은 Actuator 지표는 요청 한 번이 아니라 앱이 뜬 이후 누적된 값이라는 것을 알게 되었습니다. 요청 단위 비용을 보려면 호출 전후 값을 빼서 델타를 계산해야 한다는 것도 배웠습니다.
- Swagger UI의 Try it out은 curl 없이도 요청·응답·소요시간을 눈으로 바로 확인할 수 있어서, 실습 내내 가장 자주 쓴 도구가 되었습니다.